

Ch06 정렬 알고리즘 - 문제 정리

사진 내용을 직접 타이핑해 정리한 문서형 PDF

[보기선택형 문제]

1. [객관식] 다음 중 안정 정렬(stable sort)에 해당하는 것은?
 - ① 선택 정렬
 - ② 힙 정렬
 - ③ 삽입 정렬
 - ④ 퀵 정렬
2. [객관식] 다음 중 항상 $O(n \log n)$ 의 시간복잡도를 가지는 정렬은?
 - ① 퀵 정렬
 - ② 병합 정렬
 - ③ 버블 정렬
 - ④ 삽입 정렬
3. [객관식] 다음 중 제자리 정렬(in-place sort)이 아닌 것은?
 - ① 선택 정렬
 - ② 삽입 정렬
 - ③ 병합 정렬
 - ④ 힙 정렬
4. [객관식] 다음 중 비교 기반 정렬은?
 - ① 병합 정렬
 - ② 퀵 정렬
 - ③ 도수 정렬
 - ④ 힙 정렬
5. [객관식] 다음 중 최악의 경우 $O(n^2)$ 의 시간복잡도를 가지는 정렬은?
 - ① 병합 정렬
 - ② 힙 정렬
 - ③ 퀵 정렬
 - ④ 도수 정렬
6. [객관식] 다음 중 버블 정렬의 특징으로 옳은 것은?
 - ① 분할 정복 기반이다
 - ② 인접한 두 요소를 비교하여 교환한다
 - ③ 항상 빠른 성능을 보장한다
 - ④ 추가 메모리를 많이 사용한다

7. [객관식] 다음 중 선택 정렬의 특징으로 옳은 것은?

- ① 안정 정렬이다
- ② 입력 상태에 따라 성능이 달라진다
- ③ 비교 횟수가 항상 동일하다
- ④ 재귀를 사용한다

8. [객관식] 다음 중 삽입 정렬이 효율적인 경우는?

- ① 완전히 무작위 데이터
- ② 역순 데이터
- ③ 거의 정렬된 데이터
- ④ 매우 큰 데이터

9. [객관식] 다음 중 셸 정렬의 특징으로 옳은 것은?

- ① 항상 안정 정렬이다
- ② Gap을 이용하여 멀리 떨어진 요소를 정렬한다
- ③ 분할 정복 방식이다
- ④ 추가 메모리를 많이 사용한다

10. [객관식] 다음 중 퀵 정렬의 특징으로 옳은 것은?

- ① 항상 $O(n \log n)$ 이다
- ② 안정 정렬이다
- ③ 피벗을 기준으로 분할한다
- ④ 비교 연산을 사용하지 않는다

11. [객관식] 다음 중 병합 정렬의 특징으로 옳은 것은?

- ① 제자리 정렬이다
- ② 항상 일정한 시간복잡도를 가진다
- ③ 비교 연산을 사용하지 않는다
- ④ 힙 구조를 사용한다

12. [객관식] 다음 중 힙 정렬의 특징으로 옳은 것은?

- ① 안정 정렬이다
- ② 분할 정복 방식이다
- ③ 완전이진트리를 사용한다
- ④ 비교 연산을 사용하지 않는다

13. [객관식] 다음 중 도수 정렬의 특징으로 옳은 것은?

- ① 비교 기반 정렬이다
- ② 항상 빠르다
- ③ 값의 범위에 의존한다
- ④ 실수 데이터에 적합하다

14. [객관식] 다음 중 힙 생성(build heap)의 시간복잡도는?

- ① $O(n)$
- ② $O(n \log n)$
- ③ $O(\log n)$
- ④ $O(n^2)$

15. [객관식] 다음 중 퀵 정렬의 최악의 경우로 가장 적절한 것은?

- ① 랜덤 데이터
- ② 데이터 개수가 적은 경우
- ③ 이미 정렬된 데이터 + 나쁜 pivot 선택
- ④ 중복 데이터

16. [객관식] 다음 중 병합 정렬과 퀵 정렬의 차이로 옳은 것은?

- ① 병합 정렬은 제자리 정렬이다
- ② 퀵 정렬은 항상 안정 정렬이다
- ③ 병합 정렬은 추가 메모리를 사용한다
- ④ 퀵 정렬은 분할 정복이 아니다

17. [객관식] 다음 중 정렬 알고리즘 선택 기준으로 적절하지 않은 것은?

- ① 데이터 크기
- ② 데이터 분포
- ③ 메모리 제약
- ④ 변수 이름

18. [객관식] 다음 중 도수 정렬이 비효율적인 경우는?

- ① 데이터 개수가 많을 때
- ② 데이터 범위(k)가 매우 클 때
- ③ 중복 데이터가 많을 때
- ④ 정수 데이터일 때

19. [객관식] 다음 중 정렬 알고리즘의 안정성에 영향을 주는 요소는?

- ① 비교 방식
- ② CPU 속도
- ③ 운영체제
- ④ 컴파일러

20. [객관식] 다음 중 삽입 정렬과 선택 정렬의 차이로 옳은 것은?

- ① 선택 정렬은 안정 정렬이다
- ② 삽입 정렬은 데이터 이동이 많다
- ③ 선택 정렬은 $O(n)$ 이다
- ④ 삽입 정렬은 비교를 하지 않는다

[코드형 문제]

21. [코드문제] 다음 코드의 실행 결과는? (버블 정렬)

```
arr = [5, 3, 4, 1, 2]
for i in range(2):
    for j in range(0, len(arr)-1-i):
        if arr[j] > arr[j+1]:
            arr[j], arr[j+1] = arr[j+1], arr[j]

print(arr)
```

- ① [1, 2, 3, 4, 5]
- ② [3, 4, 1, 2, 5]
- ③ [3, 1, 2, 4, 5]
- ④ [5, 4, 3, 2, 1]

22. [코드문제] 다음 코드의 실행 결과는? (버블 정렬-비교횟수)

```
arr = [1, 2, 3, 4]
cnt = 0
for i in range(len(arr)-1):
    for j in range(len(arr)-1-i):
        cnt += 1
        if arr[j] > arr[j+1]:
            arr[j], arr[j+1] = arr[j+1], arr[j]

print(cnt)
```

- ① 3
- ② 4
- ③ 6
- ④ 10

23. [코드문제] 다음 코드의 실행 결과는? (버블 정렬-조기종료)

```
arr = [1, 2, 3, 4]
cnt = 0
for i in range(len(arr)-1):
    swapped = False
    for j in range(len(arr)-1-i):
        cnt += 1
        if arr[j] > arr[j+1]:
            arr[j], arr[j+1] = arr[j+1], arr[j]
            swapped = True

    if not swapped:
        break

print(cnt)
```

- ① 3
- ② 4
- ③ 6
- ④ 10

24. [코드문제] 다음 코드의 실행 결과는? (선택 정렬)

```
arr = [4, 2, 5, 1, 3]
for i in range(2):
    min_idx = i
    for j in range(i+1, len(arr)):
        if arr[j] < arr[min_idx]:
            min_idx = j
    arr[i], arr[min_idx] = arr[min_idx], arr[i]

print(arr)
```

- ① [1, 2, 3, 4, 5]
- ② [1, 2, 5, 4, 3]
- ③ [1, 4, 5, 2, 3]
- ④ [2, 4, 1, 5, 3]

25. [코드문제] 다음 코드가 오름차순 선택 정렬이 되기 위해 빈칸에 들어갈 값은? (선택 정렬 - 빈칸 추론)

```
arr = [3, 1, 4, 2]
for i in range(len(arr)-1):
    min_idx = i
    for j in range(i+1, len(arr)):
        if arr[j] < arr[min_idx]:
            min_idx = j
    arr[i], arr[____] = arr[____], arr[i]

print(arr)
```

- ① i, j
- ② min_idx, min_idx
- ③ j, i
- ④ i+1, min_idx

26. [코드문제] 다음 코드의 출력 결과는? (선택 정렬 - 비교 횟수)

```
arr = [5, 4, 3, 2, 1]
cnt = 0
for i in range(len(arr)-1):
    for j in range(i+1, len(arr)):
        cnt += 1

print(cnt)
```

- ① 5
- ② 10
- ③ 15
- ④ 20

27. [코드문제] 다음 코드의 출력 결과는? (삽입 정렬)

```
arr = [4, 2, 3, 1]
for i in range(1, 3):
    key = arr[i]
    j = i - 1
    while j >= 0 and arr[j] > key:
        arr[j+1] = arr[j]
        j -= 1
    arr[j+1] = key

print(arr)
```

- ① [2, 3, 4, 1]
- ② [1, 2, 3, 4]
- ③ [2, 4, 3, 1]
- ④ [4, 3, 2, 1]

28. [코드문제] 다음 코드의 출력 결과는? (삽입 정렬 - 이동 횟수)

```
arr = [5, 3, 4, 1]
move = 0
for i in range(1, len(arr)):
    key = arr[i]
    j = i - 1
    while j >= 0 and arr[j] > key:
        arr[j+1] = arr[j]
        move += 1
        j -= 1
    arr[j+1] = key

print(move)
```

- ① 3
- ② 4
- ③ 5
- ④ 6

29. [코드문제] 오름차순 삽입 정렬이 되기 위해 빈칸에 들어갈 것은? (삽입 정렬 - 빈칸 추론)

```
arr = [3, 1, 2]
for i in range(1, len(arr)):
    key = arr[i]
    j = i - 1

    while j >= 0 and arr[j] > key:
        arr[j+1] = arr[j]
        j -= 1

    arr[____] = key
print(arr)
```

- ① j
- ② j+1
- ③ i
- ④ i+1

30. [코드문제] 다음 코드의 출력 결과는? (셸 정렬)

```
arr = [8, 1, 6, 2, 5, 3]
gap = 3
for i in range(gap, len(arr)):
    temp = arr[i]
    j = i
    while j >= gap and arr[j-gap] > temp:
        arr[j] = arr[j-gap]
        j -= gap
    arr[j] = temp

print(arr)
```

- ① [1, 2, 3, 5, 6, 8]
- ② [2, 1, 3, 8, 5, 6]
- ③ [5, 1, 3, 2, 8, 6]
- ④ [1, 8, 2, 6, 3, 5]

31. [코드문제] 다음 코드의 출력 결과는? (퀵 정렬 - partition 결과)

```
arr = [6, 3, 8, 2, 5]
pivot = arr[0]
left = []
right = []
for x in arr[1:]:
    if x < pivot:
        left.append(x)
    else:
        right.append(x)

print(left, pivot, right)
```

- ① [3, 2, 5] 6 [8]
- ② [2, 3, 5] 6 [8]
- ③ [3, 8, 2, 5] 6 []
- ④ [8] 6 [3, 2, 5]

32. [코드문제] 다음 코드의 출력 결과는? (퀵 정렬 - 재귀 결과)

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[0]
    left = [x for x in arr[1:] if x < pivot]
    right = [x for x in arr[1:] if x >= pivot]

    return quick_sort(left) + [pivot] + quick_sort(right)
print(quick_sort([4, 2, 4, 1]))
```

- ① [1, 2, 4]
- ② [1, 2, 4, 4]
- ③ [4, 4, 2, 1]
- ④ 오류

33. [코드문제] 빈칸에 들어갈 값은? (퀵 정렬 - 빈칸 추론)

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[0]
    left = [x for x in arr[1:] if x < pivot]
    right = [x for x in arr[1:] if x >= pivot]

    return quick_sort(left) + [____] + quick_sort(right)
print(quick_sort([3, 1, 2]))
```

- ① arr
- ② left
- ③ pivot
- ④ right

34. [코드문제] 다음 코드의 출력 결과는? (퀵 정렬 - 재귀 결과)

```

arr = [6, 4, 8, 3, 5]
pivot = arr[0]
left = 1
right = len(arr) - 1
while left <= right:
    while left <= right and arr[left] <= pivot:
        left += 1
    while left <= right and arr[right] >= pivot:
        right -= 1
    if left < right:
        arr[left], arr[right] = arr[right], arr[left]
arr[0], arr[right] = arr[right], arr[0]
print(arr)

```

- ① [5, 4, 3, 6, 8]
- ② [3, 4, 5, 6, 8]
- ③ [4, 3, 5, 6, 8]
- ④ [6, 4, 8, 3, 5]

35. [코드문제] 다음 코드의 출력 결과는? (병합 정렬 - merge 과정)

```

left = [1, 4, 7]
right = [2, 3, 6]
result = []
i = j = 0
while i < len(left) and j < len(right):
    if left[i] <= right[j]:
        result.append(left[i])
        i += 1
    else:
        result.append(right[j])
        j += 1
result += left[i:]
result += right[j:]
print(result)

```

- ① [1, 2, 3, 4, 6, 7]
- ② [1, 4, 7, 2, 3, 6]
- ③ [2, 3, 6, 1, 4, 7]
- ④ [1, 2, 4, 3, 6, 7]

36. [코드문제] 다음 코드의 출력 결과는? (병합 정렬 - 안정성)

```

left = [(2, 'A'), (3, 'B')]
right = [(2, 'C'), (4, 'D')]
result = []
i = j = 0
while i < len(left) and j < len(right):
    if left[i][0] <= right[j][0]:
        result.append(left[i])
        i += 1
    else:
        result.append(right[j])
        j += 1
result += left[i:]
result += right[j:]
print(result)

```

- ① [(2, 'A'), (2, 'C'), (3, 'B'), (4, 'D')]
- ② [(2, 'C'), (2, 'A'), (3, 'B'), (4, 'D')]
- ③ [(2, 'A'), (3, 'B'), (2, 'C'), (4, 'D')]
- ④ 오류

37. [코드문제] 다음 코드의 출력 결과는? (힙 정렬 - down heap 1회)

```

arr = [3, 8, 7, 1, 5]
parent = 0
child = 1
if child + 1 < len(arr) and arr[child] < arr[child+1]:
    child += 1
if arr[parent] < arr[child]:
    arr[parent], arr[child] = arr[child], arr[parent]
print(arr)

```

- ① [8, 3, 7, 1, 5]
- ② [7, 8, 3, 1, 5]
- ③ [3, 8, 7, 1, 5]
- ④ [8, 7, 3, 1, 5]

38. [코드문제] 다음 코드의 출력 결과는? (힙 정렬 - down heap 1회)

```
arr = [4, 10, 3, 5, 1]
n = len(arr)
# heapify (최대 힙 구성)
for i in range(n//2 - 1, -1, -1):
    parent = i
    child = 2 * parent + 1
    if child + 1 < n and arr[child] < arr[child+1]:
        child += 1
    if arr[parent] < arr[child]:
        arr[parent], arr[child] = arr[child], arr[parent]

print(arr)
```

- ① [10, 5, 3, 4, 1]
- ② [10, 4, 3, 5, 1]
- ③ [4, 10, 3, 5, 1]
- ④ [5, 10, 3, 4, 1]

39. [코드문제] 다음 코드의 출력 결과는? (도수 정렬 - 도수 배열)

```
arr = [2, 0, 1, 2, 1, 0, 2]
count = [0] * 3
for x in arr:
    count[x] += 1
print(count)
```

- ① [2, 2, 3]
- ② [3, 2, 2]
- ③ [0, 1, 2]
- ④ [2, 3, 2]

40. [코드문제] 다음 코드의 출력 결과는? (도수 정렬 - 누적 도수)

```
count = [2, 2, 3]
for i in range(1, len(count)):
    count[i] += count[i-1]
print(count)
```

- ① [2, 2, 3]
- ② [2, 4, 7]
- ③ [7, 5, 3]
- ④ [0, 2, 4]